

EvoCrypt: An Upgradable Cryptographic Coprocessor for the Ethereum Virtual Machine

Qi Liu¹, Qianhong Wu¹, Bo Qin²

¹School of Cyber Science and Technology, Beihang University, Beijing, 100191, China.

²School of Information, Renmin University of China, Beijing, 100872, China.

Abstract

Smart contracts frequently rely on cryptographic algorithms, yet implementing complex cryptographic computation in Solidity expands it into large sequences of EVM instructions, resulting in substantial execution overhead and gas consumption. Precompiled contracts reduce these costs by exposing client-native routines at fixed addresses, but they provide no runtime mechanism for evolving an algorithm independently of client releases and coordinated network upgrades. We present EvoCrypt, an upgradeable cryptographic coprocessor that exposes a stable contract-facing interface while executing replaceable WebAssembly modules in a controlled client-side runtime. EvoCrypt distributes version metadata on-chain and deterministically activates an installed module at a designated block height, allowing participating nodes to switch algorithm versions consistently. Each invocation is charged according to a deterministic algorithm-level gas schedule associated with the activated version, thereby avoiding the cost of expanding the algorithm into EVM instructions while retaining explicit resource accounting. We implement EvoCrypt in Geth and evaluate it on a 20 node private Clique network configured with zero-period block production to isolate system-level upgrade overhead from the block interval. Measured *network-wide upgrade duration* is the Geth-side time to finish upgrade handling on all evaluated nodes—from RPC submission through receipt observation, event handling, and node-local WASM preparation—and excludes network propagation and consensus delay; under this definition the prototype completes upgrades within 0.54–2.34 ms across the eight evaluated algorithms. EvoCrypt achieves execution latency comparable to Solidity implementations on lightweight algorithms, with up to 48% lower latency on computation-heavy workloads.

Keywords: blockchain, smart contracts, upgrade, WebAssembly

1 Introduction

Blockchain systems are increasingly used in digital finance, supply-chain management, digital identity, and trusted data exchange, and have evolved from narrowly scoped distributed ledgers into programmable trusted computing platforms [1, 2]. Smart contracts provide the core mechanism for this programmability: they execute predefined rules for asset transfer, authorization, state update, and multi-party coordination within a deterministic blockchain execution environment [3, 4]. These operations commonly depend on cryptographic functions for identity authentication, data integrity, digital signatures, and privacy protection, so smart contracts require ways to invoke or execute cryptographic algorithms to support trustworthy on-chain business logic. At the same time, advances in quantum computing challenge the long-term security assumptions of deployed public-key cryptography [5, 6], and the standardization of post-quantum cryptographic algorithms has made post-quantum migration an important requirement for long-term blockchain security [7]. Because many on-chain workflows expose cryptographic capabilities through smart contracts, this migration requires not only support for new cryptographic algorithms in the underlying blockchain client but also timely access to these capabilities from smart contracts. Providing extensible and evolvable support for contract-facing cryptographic algorithms has therefore become an important problem for blockchain systems designed for long-term security evolution.

Ethereum currently offers two common routes for executing cryptographic logic. A developer can implement an algorithm directly in Solidity, which keeps the logic flexible and deployable at the contract layer but executes the computation through Ethereum Virtual Machine (EVM) instructions [3, 4]. That path imposes two recurring obstacles for contract-facing cryptography. The first is an engineering obstacle: complex arithmetic, byte-oriented workloads, and correctness-sensitive logic must be expressed as EVM bytecode, which makes non-trivial cryptographic implementations difficult to develop, audit, and maintain. The second is an economic obstacle: gas prices computation at instruction level [3, 8], so a cryptographic algorithm expanded into a long EVM instruction sequence is charged for every opcode it executes. We refer to this effect as *gas amplification*: the gas of an in-contract implementation grows with its expanded instruction count rather than with the algorithmic work performed, and computationally intensive Solidity implementations therefore incur substantial execution latency and gas overhead. Alternatively, Ethereum clients can expose selected algorithms as Precompiled Contracts, which execute native client code behind predefined EVM interfaces [9]. This route lowers execution cost for algorithms already fixed in the protocol catalog, but it does not provide a general contract-layer way to add arbitrary new cryptographic workloads without a client release, and it binds each exposed routine to the client software. Updating a Precompiled Contract normally requires modifying client source code, rebuilding the client, and coordinating deployment across nodes.

This split leaves a capability gap relative to those Solidity obstacles. When a required algorithm is not available as a precompile, in-contract Solidity remains the default deployment path, yet it remains poorly matched to expensive cryptographic computation. Precompiled Contracts improve execution efficiency for a fixed set of

routines but are difficult to update independently of the client release process and do not by themselves address algorithm evolution requirements such as post-quantum migration. More generally, smart contract engineering already faces persistent challenges around correctness, cost, and maintainability [2]. Prior work has examined live-upgradable blockchain clients and automated smart-contract patching [10, 11], showing the broader relevance of upgradeability in blockchain software. For cryptographic algorithms, however, the challenge is to support algorithm replacement without changing the semantics of historical execution or allowing different nodes to select different algorithm versions at the same block height. A process-local plugin or shared-library boundary ties execution to platform-specific native binaries and weakens the goal that every node apply the same algorithm semantics at the same block. Existing approaches do not directly provide a low-intrusion mechanism for replacing contract-facing cryptographic implementations with a uniform, portable module payload while preserving deterministic activation across nodes.

This paper proposes a general Cryptographic Coprocessor architecture for smart-contract-enabled blockchain clients. The coprocessor decouples the contract-facing cryptographic algorithm interface from the WASM module implementation managed by the client. Smart contracts invoke a stable EVM-facing interface to obtain cryptographic results without embedding the algorithm in contract bytecode, while the client maintains replaceable WASM algorithm modules behind that interface. The upgrade path combines on-chain coordination with off-chain asynchronous preparation: an upgrade transaction records metadata and emits an event, while client-side listeners verify, compile, and instantiate the target WASM module outside the EVM execution critical path of that transaction. To prevent nodes from switching versions according to local readiness time, the protocol separates asynchronous upgrade preparation from deterministic activation through an Activation Block. Because algorithm execution is moved behind the coprocessor boundary, the invocation path charges a deterministic algorithm-level gas schedule registered in the on-chain version metadata instead of metering an expanded EVM instruction sequence, so contract-facing cryptographic calls are not subject to gas amplification.

The main contributions of this article are summarized as follows.

- Firstly, we design a Cryptographic Coprocessor that separates contract-facing cryptographic algorithm interfaces from WASM module implementations, allowing algorithm implementations to be managed and replaced independently of the core EVM execution logic while using a narrower runtime boundary than process-local plugin loading.
- Secondly, we design an upgrade protocol that combines on-chain event coordination, off-chain asynchronous WASM preparation, and block-height-based deterministic activation.
- Finally, we implement a prototype of EvoCrypt and evaluate it on a 20-node private blockchain network. Experimental results show that algorithm-level gas pricing decouples per-invocation cost from the expanded EVM instruction count, thereby avoiding the opcode-level gas amplification observed in Solidity executions; the registered prices in the evaluated prototype are experimental parameters, so the results

demonstrate the pricing mechanism and the resulting capability extension rather than a calibrated magnitude of gas savings.

The resulting claims are intentionally bounded: the evaluation concerns contract-facing cryptographic algorithms under the evaluated scenarios, does not claim that unrelated EVM transactions are unaffected during the upgrade preparation window, does not set up a separate consistency experiment, and does not treat the WASM sandbox as a formal security proof.

2 Background

2.1 Smart Contracts

Ethereum smart contracts are programs that run on contract accounts. A contract account is identified by an address, its code hash points to EVM bytecode stored in the state database, and a transaction or message call to that address triggers bytecode execution in the EVM [3, 12, 13]. Gas is the protocol-level budget for this execution. The protocol charges for opcode execution, memory use, state access, and transaction data, so deterministic execution is coupled to an resource-pricing mechanism [3, 8].

Cryptographic algorithms can be exposed to smart contracts in several ways. A Solidity contract implementation is flexible because it can be deployed and updated at the contract layer, but complex arithmetic or byte-oriented computation expands into long EVM instruction sequences, and per-opcode pricing can therefore amplify both latency and gas cost. A Precompiled Contract provides a reserved EVM-facing address whose execution is dispatched to native client code. The elliptic-curve precompiles introduced by EIP-196 and EIP-197, together with EIP-1109’s call-cost discussion, illustrate this pattern: the interface is stable at the protocol boundary, while the implementation and gas schedule are fixed by client and protocol rules [9, 14–16]. This improves execution efficiency, but adding or replacing an algorithm usually follows the client release path rather than an application-level deployment path.

2.2 Live Upgrade

A blockchain client receives and validates blocks, maintains local state, executes transactions, and exposes RPC interfaces. In Ethereum, the execution client handles transaction validation, state management, and EVM execution, while the consensus client handles fork choice and finality coordination [17, 18]. Updating functionality inside an execution client is therefore different from updating an ordinary application library: a faulty or inconsistently adopted update can affect transaction replayability and the semantics that different nodes apply to the same block.

Traditional client upgrades rely on software distribution, local deployment, and process restart. Dynamic software update (DSU) mechanisms reduce this operational cost in general distributed systems, and live-upgradable blockchain clients show that runtime replacement is also relevant in blockchain software [10, 19, 20]. For contract-facing cryptographic algorithms, however, runtime upgrade must also preserve a deterministic version boundary. A node must not decide the active cryptographic implementation from local wall-clock time or local preparation completion alone,

because the same contract call at the same block height must resolve to the same algorithm version under the same chain view.

2.3 WebAssembly

WebAssembly defines a portable low-level bytecode format intended for compact representation and efficient execution under a host runtime [21]. A WASM module executes with explicit imports, exports, and linear memory. In this work, WASM is used as the dynamically loaded algorithm artifact, replacing the earlier process-local plugin boundary with a bytecode payload that each node can decode, hash, compile, instantiate, and invoke through a controlled runtime interface.

This runtime boundary is useful for cryptographic algorithm modules because it separates the host client from the module’s internal implementation language and binary layout. EvoCrypt treats the WASM bytecode as the authoritative algorithm payload: every node verifies the same bytes through the module hash recorded in on-chain metadata, then compiles and invokes that artifact through a controlled runtime interface. For a given algorithm version, the same WASM module and the same ABI-encoded input therefore produce the same coprocessor output only when every node executes that module on the same WASM virtual machine implementation and version. The Geth prototype pins this boundary to wazero v1.12.0 with a fixed import policy and compilation cache layout, so heterogeneous operating systems and hardware still replay identical coprocessor semantics at a given block height once version selection is fixed. This portability is a principal reason to prefer WASM over process-local native plugins, whose binaries vary by platform. Sandbox isolation remains an engineering control rather than a formal security theorem; production deployments must enforce a single approved runtime name and version across validators in addition to the shared bytecode hash. The evaluation in this paper is limited to the implemented Geth prototype and the wazero v1.12.0 runtime path.

3 EvoCrypt Design

The Cryptographic Coprocessor provides runtime-upgradable WASM implementations for contract-facing cryptographic algorithms without turning the entire blockchain client into a dynamically updated component. We specify the design by three objects reused by the protocols below: a version metadata record $\mu_{a,v}$, a height-based version selector $v^*(a, b)$, and an algorithm-level gas schedule $G(a, v, x)$. Algorithm logic is shipped as portable WASM bytecode rather than per-node native binaries, so heterogeneous clients execute the same module artifact under the same selection rule. Ordinary transactions continue through the EVM under unchanged opcode-level pricing. Only selected cryptographic implementations behind a stable contract-facing interface are prepared, loaded, and selected as replaceable WASM modules.

3.1 Overview

Figure 1 shows the main modules. The Execution Engine executes ordinary EVM transactions. Identity Hooks recognize calls to the Management Contract entry point

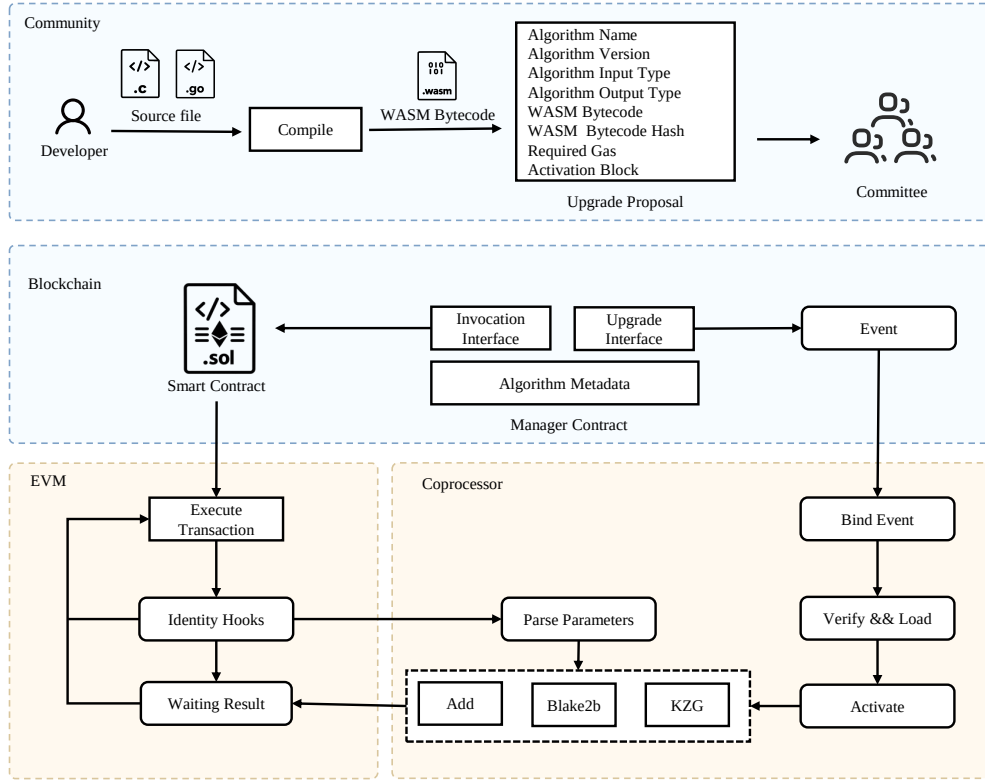


Fig. 1: Workflow of the Upgradable Cryptographic Coprocessor.

used by the prototype and route only those calls into the coprocessor boundary. The Cryptographic Coprocessor manages ABI decoding, algorithm lookup, version selection, gas metadata, and WASM invocation. The Upgrade Handler listens for on-chain upgrade events and prepares node-local artifacts. The WASM Runtime compiles and instantiates modules under a controlled import/export and memory boundary, and WASM Modules contain the replaceable algorithm implementations.

The on-chain layer contains the Management Contract, event log, version metadata, and Activation Block. The Management Contract abstracts the CodeStorage contract used in the prototype: its invocation interface exposes the contract-facing call entry, and its upgrade interface publishes the metadata record defined in (1). Clients derive the module hash from the decoded artifact during local preparation. The client-side layer contains the Execution Engine, Identity Hooks, Cryptographic Coprocessor, Upgrade Handler, WASM Runtime, and local module workspace. This split keeps the chain as the authoritative coordination surface while leaving module preparation as node-local work. Section 3.2 defines the metadata record and the upgrade protocol that publishes it. Section 3.3 defines the invocation path and the gas schedule that charges a call to the selected version.

3.2 Upgrade Protocol

An upgrade starts from an on-chain proposal. In the versioned path, a committee-authorized proposer submits the proposal through `uploadCodeVersion`, and the Management Contract admits it only after the committee reaches consensus on its contents. During transaction execution, the contract stores the admitted record and emits a versioned upgrade event. The EVM path does not compile or instantiate the WASM module; it records the authoritative metadata and publishes an event that clients can observe.

Definition 1 (Version metadata) Let a be an algorithm name and v a version number. An admitted upgrade is the metadata record

$$\mu_{a,v} = (a, v, C_{a,v}, h_{a,v}, g_{a,v}^{(0)}, \lambda_{a,v}, \tau_{a,v}^{\text{in}}, \tau_{a,v}^{\text{out}}, H_{a,v}), \quad (1)$$

where $C_{a,v}$ is the compiled WASM bytecode, $h_{a,v} = \text{Keccak256}(C_{a,v})$ is the module hash, $g_{a,v}^{(0)}$ is the initial gas fee, $\lambda_{a,v}$ is the gas slope parameter, $\tau_{a,v}^{\text{in}}$ and $\tau_{a,v}^{\text{out}}$ are the ABI input and output type descriptors, and $H_{a,v}$ is the Activation Block height. The Management Contract stores an encoding of these fields. Each node recomputes $h_{a,v}$ from the decoded bytecode during preparation and binds that hash to the prepared version.

Table 1 summarizes the components of $\mu_{a,v}$. Write $\mathcal{V}_a$ for the set of versions already registered for algorithm a . The chain is the sole source of $\mu_{a,v}$; the upgrade event carries only $\langle a, v, H_{a,v} \rangle$ as a notification index.

Table 1: Components of the version metadata record $\mu_{a,v}$.

Symbol	Field	Role
a	algorithm name	Identifier of the contract-facing algorithm.
v	algorithm version	Monotone version number within $\mathcal{V}_a$.
$C_{a,v}$	WASM bytecode	Compiled module artifact executed by the runtime.
$h_{a,v}$	module hash	Keccak-256 digest of $C_{a,v}$, bound at preparation.
$g_{a,v}^{(0)}$	initial gas fee	Constant term of the invocation schedule (3).
$\lambda_{a,v}$	gas slope parameter	Per-byte coefficient of the invocation schedule.
$\tau_{a,v}^{\text{in}}$	input type	ABI descriptor of the call-stage inner input.
$\tau_{a,v}^{\text{out}}$	output type	ABI descriptor of the returned result.
$H_{a,v}$	activation height	First block at which version v may become active.

Each client runs an Event Listener for Management Contract upgrade events. When a listener observes $\langle a, v, H \rangle$, it recovers $\mu_{a,v}$ through `getVersionInfo`, checks that the event and stored record agree on v and $H_{a,v}$, and delegates preparation to the Upgrade Handler. The handler decodes $C_{a,v}$, computes $h_{a,v}$, asks the WASM Runtime to compile and instantiate the module, validates the required `execute` export, and records (a, v) as a prepared version.

This workflow separates on-chain coordination from node-local preparation. The chain provides one admitted record $\mu_{a,v}$, while each node independently verifies and

Algorithm 1: Activation-Block-based WASM upgrade protocol.

Input: Metadata $\mu_{a,v}$ as in (1); node set $\mathcal{N}$

Output: Prepared set updated on every $n_i \in \mathcal{N}$

```

1: committee-authorized proposer submits uploadCodeVersion( $\mu_{a,v}$ )
2: Management Contract stores  $\mu_{a,v}$ ; emits event  $\langle a, v, H_{a,v} \rangle$ 
3: for all  $n_i \in \mathcal{N}$  do
4:   observe  $\langle a, v, H_{a,v} \rangle$ ; fetch  $\mu'_{a,v}$  from Management Contract
5:   if event fields disagree with  $\mu'_{a,v}$  then
6:     reject  $(a, v)$ 
7:   else
8:     decode  $C_{a,v}$ ; set  $h_{a,v} \leftarrow \text{Keccak256}(C_{a,v})$ 
9:     repeat
10:      compile and instantiate; validate execute export
11:    until preparation succeeds
12:    persist  $h_{a,v}$  and  $\mu_{a,v}$ ; mark  $(a, v)$  prepared
13:   end if
14: end for

```

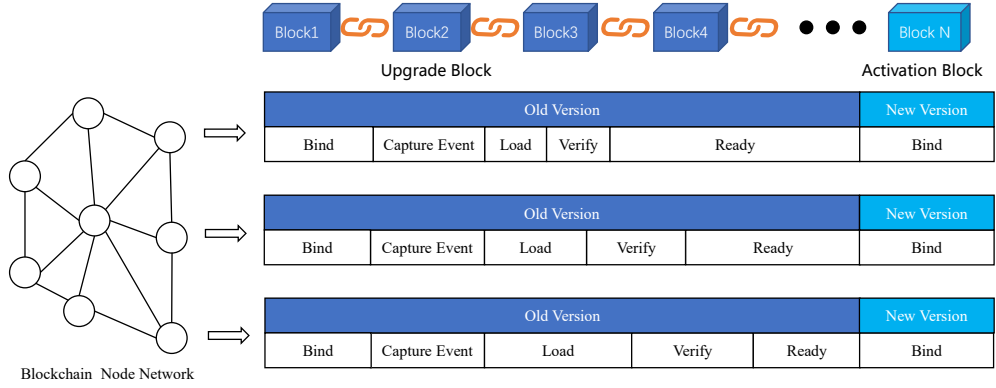


Fig. 2: Node-local preparation and deterministic network-wide activation of an EvoCrypt upgrade.

prepares the same artifact. Under the mandatory-upgrade policy, a node that observes a valid upgrade event must complete preparation of (a, v) before treating that version as callable. A failed attempt retries from the contract record and does not roll back the already executed upgrade transaction. The upgrade transaction therefore does not wait for WASM compilation. Algorithm 1 summarizes the protocol.

Asynchronous preparation would break execution consistency if nodes switched versions as soon as local compilation finished. Two nodes may observe the same chain but complete preparation at different wall-clock times. The coprocessor therefore separates preparation from activation through $H_{a,v}$. Figure 2 shows this two-phase timeline on the blockchain node network.

The Upgrade Block is the block that includes the admitted `uploadCodeVersion` transaction and emits $\langle a, v, H_{a,v} \rangle$. After that inclusion, each node independently captures the event, recovers $\mu_{a,v}$, and completes the off-chain Load–Verify–Ready sequence of Algorithm 1. Invocations in this interval still use the previously selected version: local readiness does not change which implementation is served. When the chain reaches the Activation Block $H_{a,v}$, nodes that share the same chain view bind version v together. The next definition makes this height-based switch precise.

Definition 2 (Active version) Let $\mathcal{V}_a$ be the registered version set of algorithm a , and let b be the execution block height. The version used at height b is

$$v^*(a, b) = \arg \max_{v \in \mathcal{V}_a} \{(H_{a,v}, v) \mid H_{a,v} \leq b\}. \quad (2)$$

The $\arg \max$ is lexicographic on the pair $(H_{a,v}, v)$: among versions whose activation height does not exceed b , the rule prefers the latest activation height and, for a tied height, the larger version number. If the feasible set is empty, no version is active and the invocation fails with a missing-version error.

Equation (2) makes activation a function of chain height rather than local readiness. Given the same chain view and the same $\{\mu_{a,v}\}_{v \in \mathcal{V}_a}$, every node computes the same $v^*(a, b)$. The protocol is a forced upgrade: once $v^*(a, b)$ is defined, a node must execute that version and must not serve an obsolete implementation. If $(a, v^*(a, b))$ is not yet prepared when a call arrives or when height b is processed, the node reloads $\mu_{a, v^*(a, b)}$ from the Management Contract and completes validation, compilation, and instantiation before proceeding.

3.3 Invocation Protocol

The Invocation Protocol exposes a uniform contract-facing interface. A caller supplies the algorithm name a and an ABI-encoded input x that conforms to $\tau_{a, v^*(a, b)}^{\text{in}}$. The contract does not depend on the artifact location, implementation language, or local loading state. It depends only on the EVM-facing boundary and on the ABI contract of the selected version. If $v^*(a, b)$ is not yet prepared, the coprocessor reloads $\mu_{a, v^*(a, b)}$ and completes compilation before execution, rather than redirecting the call to an obsolete implementation.

Inside the client, the EVM integration boundary recognizes the `CodeStorage` target and the `callFunc` selector, then parses the outer calldata and normalizes a . The inner input x remains ABI-encoded across the host–runtime boundary: a dynamically upgraded WASM module receives these bytes through its `execute` export and returns an ABI-encoded result, whereas a built-in routine may decode and re-encode the values in the host. In both cases the selected result is returned to the EVM call frame. Replacing an implementation therefore does not change the contract-facing call shape, provided that $\tau_{a, v}^{\text{in}}$ and $\tau_{a, v}^{\text{out}}$ remain compatible.

At the data plane, a caller encodes the typed parameters with $\tau_{a, v^*(a, b)}^{\text{in}}$, wraps them in a `callFunc(string, bytes)` envelope, and issues a read-only `STATICCALL` to `CodeStorage`. Calls whose selector is `callFunc` enter the invocation path; other

Algorithm 2: Contract-to-coprocessor invocation protocol.

Input: Caller contract, algorithm name a , typed input p , block height b , gas budget g

Output: ABI-encoded result y or explicit error

```
1: build callFunc calldata  $d$  from  $p$ ; issue STATICCALL to CodeStorage with gas  $g$ 
2: if target is not CodeStorage callFunc then
3:   continue ordinary EVM or management dispatch; return
4: end if
5: if calldata parsing fails then
6:   return ABI error
7: end if
8: parse  $(a, x)$  from  $d$ ; set  $v \leftarrow v^*(a, b)$  by (2)
9: if no applicable version  $v$  then
10:  return missing-version error
11: end if
12: if  $(a, v)$  not prepared then
13:  repeat
14:    forcibly reload  $\mu_{a,v}$  and prepare  $(a, v)$ 
15:  until  $(a, v)$  prepared
16: end if
17:  $G_{\text{req}} \leftarrow G(a, v, x)$  by (3)
18: if  $g < G_{\text{req}}$  then
19:  return out-of-gas error
20: end if
21: charge  $G_{\text{req}}$ ;  $y \leftarrow \text{WASM}(a, v, x)$ 
22: return  $y$ , or runtime error on failure
```

`CodeStorage` selectors continue through metadata or management dispatch; all other addresses follow ordinary EVM routing. The dispatcher parses (a, x) , computes $v^*(a, b)$ by (2), and blocks on forced preparation when the mandatory-upgrade policy applies. It then charges the deterministic fee defined below, executes the selected WASM module, and returns the ABI-encoded output. Invalid calldata, a missing version, insufficient gas, or a runtime failure surface as explicit errors. Algorithm 2 summarizes this flow.

Definition 3 (Algorithm-level gas schedule) Each algorithm version uses an independent pricing rule. A call with inner input x against version v of algorithm a is charged

$$G(a, v, x) = g_{a,v}^{(0)} + \lambda_{a,v} \cdot |x|, \quad (3)$$

where $|x|$ is the byte length of the call-stage inner input, $g_{a,v}^{(0)}$ is the initial gas fee, and $\lambda_{a,v}$ is the gas slope parameter registered in $\mu_{a,v}$. The charged amount does not depend on node-local CPU time or on the number of WASM instructions executed inside the module.

Equation (3) is what removes opcode-level gas amplification from the invocation path. The fee is computed from the selected version's registered coefficients and from

$|x|$, not from metering the module’s internal execution, and it is identical for every node that resolves the same $v^*(a, b)$ on the same input. The pair $(g_{a,v}^{(0)}, \lambda_{a,v})$ is therefore an explicit per-version policy: it can reflect the algorithm’s resource class rather than the expanded instruction count of an equivalent EVM implementation. The algorithm-level schedule is charged only on the `callFunc` path. Upgrade and meta-data transactions remain under ordinary transaction pricing. The coefficients used in Section 5 are experimental parameters; they demonstrate this charging mechanism rather than a calibrated economic optimum.

4 Security Analysis

4.1 Attack Circumstances

The security analysis focuses on attack circumstances introduced by cryptographic algorithms that are upgraded at runtime and invoked from contracts. Upgrade submission is restricted to committee-authorized proposers (Section 4.2), so the analysis distinguishes two adversary positions. An external attacker may craft transaction inputs and exert timing pressure around node-local upgrade preparation but cannot register artifacts or metadata fields. A faulty or compromised admitted proposer may influence uploaded artifacts and metadata fields. We also consider erroneous upgrade artifacts and local preparation failures because they can lead to the same safety outcomes as adversarial inputs. The analysis does not model a compromised Geth binary, a broken consensus protocol, a captured committee, or cryptographic primitives whose mathematical assumptions are false.

Malicious or malformed WASM artifacts. A faulty or compromised admitted proposer may submit a WASM module that is malformed, omits the required entry point, imports unauthorized host functions, declares unsafe memory behavior, or attempts to exploit the WASM runtime boundary. This scenario targets execution isolation: the module is dynamically supplied after client deployment, so the client cannot rely on source-level review or static compilation into the Geth binary.

Inconsistent upgrade metadata. An attacker or faulty proposer may publish metadata whose version, Activation Block, gas parameters, ABI type descriptors, event projection, or encoded artifact are inconsistent with the intended upgrade. Such inconsistencies can cause version confusion, incorrect ABI decoding, incorrect gas charging, or preparation of an artifact that does not correspond to the version later selected during EVM execution.

Resource-exhaustion attempts. Dynamic algorithm loading creates denial-of-service risks. A module may be large, slow to compile, memory intensive, or designed to run for a long time. A low gas setting may underprice an expensive cryptographic operation, and large uploaded artifacts may increase calldata, storage, propagation, and data-availability pressure.

Asynchronous readiness divergence. Nodes may observe the same on-chain upgrade event but complete WASM validation, compilation, instantiation, and local persistence at different wall-clock times. A node may also fail to prepare the required artifact. If local readiness time or local wall-clock time were used for version selection,

nodes executing the same block could resolve the same contract-facing call to different algorithm implementations.

4.2 Security Solutions

The proposed coprocessor addresses these attack circumstances through committee-gated upgrade admission, sandboxed WASM execution, authoritative metadata recovery, deterministic resource accounting, and Activation-Block-based version selection. These controls reduce the security boundary exposed by dynamic upgrades, but they are not a formal proof of sandbox security against all adversarial modules, governance failures, or misconfigured deployments.

Committee-gated upgrade admission. The upgrade interface is not open to arbitrary accounts. The Management Contract admits a versioned proposal only through the committee-authorized path: a proposal becomes effective only after the committee reaches consensus on the algorithm name, version, encoded artifact, invocation gas metadata, ABI descriptors, and Activation Block height. This access restriction removes the unauthorized-proposer cases from the malicious-artifact and metadata-inconsistency circumstances, because an outside attacker cannot register an artifact, a gas price, or an activation height without committee approval. The residual risk is a faulty, compromised, or captured committee together with misconfigured admission rules; these governance failures remain outside the analyzed boundary, and a production deployment would require concrete committee rotation, quorum, and key-management rules.

Sandboxed WASM execution. WASM narrows the execution boundary compared with a process-local plugin or shared-library interface. The module runs inside a WASM Runtime with linear memory, explicit exports, and validated imports. In the prototype, modules are invoked through a fixed `execute` export, and imported memories or unauthorized host imports are rejected. This design reduces direct access from the algorithm module to Geth process memory, local files, system networking, and host functions that are not explicitly exposed. A production deployment would still require stronger artifact admission policies, runtime diversity testing, reproducible builds, and governance rules for approving cryptographic modules.

Authoritative metadata recovery. The Management Contract is treated as the authoritative source of upgrade metadata. The emitted event is used as a notification and index, while the Event Listener recovers the full metadata through `getVersionInfo` and checks the observed event fields against the contract state. The decoded WASM bytes are hashed locally and bound to the prepared version metadata, and the ABI descriptors determine how inputs and outputs are decoded and encoded. This binding reduces the risk that a node prepares an artifact or ABI interface different from the version selected during execution.

Deterministic resource accounting. Invocation gas is determined by the algorithm-level schedule (3) registered in $\mu_{a,v}$, not by EVM metering of the module’s execution logic. Every node resolving the same $v^*(a, b)$ on the same inner input x evaluates the same pair $(g_{a,v}^{(0)}, \lambda_{a,v})$ from the Management Contract, so the consistency of gas charging reduces to the consistency of the contract state, which the underlying consensus protocol already provides: nodes cannot disagree on $G(a, v^*(a, b), x)$

without disagreeing on the chain itself. The design further bounds module memory, applies an execution budget, validates imports and required exports, and separates module preparation from the upgrade transaction’s EVM execution path. These measures reduce the immediate attack surface of dynamic loading. They do not fully solve economic pricing: the current prototype invocation gas values are experimental parameters, and future work must calibrate call-path gas pricing against execution cost rather than treating upgrade-upload transaction cost as part of the coprocessor pricing model. Accordingly, the evaluation in Section 5 demonstrates the pricing mechanism itself, namely that invocation cost is decoupled from the expanded EVM instruction count, together with the metered Solidity comparison side; the claimed benefit is this capability extension, not a validated magnitude of gas savings.

Forced upgrade at the Activation Block. Upgrade consistency is treated as a protocol security property. Version selection follows (2): it depends on the execution block height rather than local wall-clock time or local readiness time. Therefore, nodes with the same chain view compute the same $v^*(a, b)$. Once that version applies, a node must not silently fall back to an obsolete implementation. If local preparation is incomplete when the Activation Block is reached or when a requiring call is executed, the node forcibly reloads $\mu_{a, v^*(a, b)}$ from authoritative metadata and completes preparation before serving the new version. This paper does not set up a separate consistency experiment; the claim remains a protocol analysis rather than node-level evidence for state-root equality, arbitrary network safety, or behavior when forced preparation cannot complete.

5 Evaluation

The evaluation asks whether the proposed coprocessor can be used as a practical execution path for upgradable contract-facing cryptographic algorithms. Following the target outline, we focus on upgrade efficiency and execution efficiency. Upgrade consistency is discussed as a protocol security property in Section 4.2 and is not set up as a separate experiment.

The prototype used in these experiments was implemented in Geth by adding the CodeStorage interface, EVM call recognition for the Management Contract entry point, the Cryptographic Coprocessor dispatcher, the activation service, version metadata storage, and the WASM Runtime path. The invocation path recognizes CodeStorage calls, calculates required gas from the registered metadata, parses the outer ABI envelope, selects the applicable algorithm version for the execution block height, passes the inner ABI-encoded input to the selected WASM module, and returns its ABI-encoded output to the EVM.

The upgrade path is event-driven. `uploadCode` supports the legacy unversioned path, while `uploadCodeVersion` carries version and activation metadata. The Event Listener observes the emitted upgrade event, and the activation service decodes the uploaded WASM artifact, computes its hash, records runtime metadata, compiles and instantiates the module through the WASM Runtime, and persists the prepared-version metadata. The WASM Runtime is wazero v1.12.0 and uses a memory limit, an execution budget, import validation, and an `execute` export check. These checks define

implementation safeguards for the evaluated prototype boundary, not a substitute for a complete production security policy.

5.1 Experimental Setup

Experiments were conducted on a Geth-based private Clique network using a modified Geth v1.17.5-unstable, built with Go 1.25.7. Solidity contracts were compiled with solc 0.8.26; WASM modules were built with TinyGo 0.42.0. Host hardware specifications remain to be recorded for full reproducibility.

We report *network-wide upgrade duration* as control-client observable Geth-side setup time from upgrade submission until all evaluated nodes finish upgrade handling and the new algorithm is callable. This interval includes RPC submission, receipt observation, event handling, and node-local WASM preparation on each node, but excludes network propagation beyond the measured RPC path, consensus delay, and block-production waiting. With `period=0`, the network does not enforce an inter-block interval, so the reported duration reflects client-side upgrade processing rather than Clique block spacing.

Table 2 summarizes the eight evaluated contract-facing algorithms, their implementation artifact sizes on the three benchmark paths, and the cryptographic role of each workload. Go (B) reports the byte size of the Native reference implementations; WASM (B) reports the uploaded module payloads compiled from the same reference logic; Solidity (B) reports on-chain benchmark contract source size before deployment. For Dh2048Secret and PedersenCommit, the listed Solidity sizes correspond to algorithm-specific wrappers that link a shared modular-arithmetic library (8,524 B); deployed bytecode and invocation `gasEstimate` in Section 5.3 reflect the fully linked contracts. Add is a non-cryptographic baseline; the remaining workloads cover hash functions, password-based key derivation, key exchange, commitments, signature verification, and finite-field polynomial multiplication.

Table 2: Information on the evaluated algorithms.

Algorithm	Go (B)	WASM (B)	Solidity (B)	Role
Add	156	11,287	181	Non-cryptographic baseline
Sha256	210	68,288	219	Cryptographic hash
Blake2bSum256	4,911	8,724	5,340	Cryptographic hash
Pbkdf2Sha256	1,377	72,580	2,269	Key derivation
Dh2048Secret	1,595	45,919	473 [†]	Key exchange
PedersenCommit	1,871	95,780	645 [†]	Commitment scheme
SchnorrVerify	2,171	95,960	962	Signature verification
PolynomialMul	790	7,987	953	Algebraic operation

[†] Solidity source size for the algorithm contract only; modular arithmetic is provided by the shared library linked at deployment.

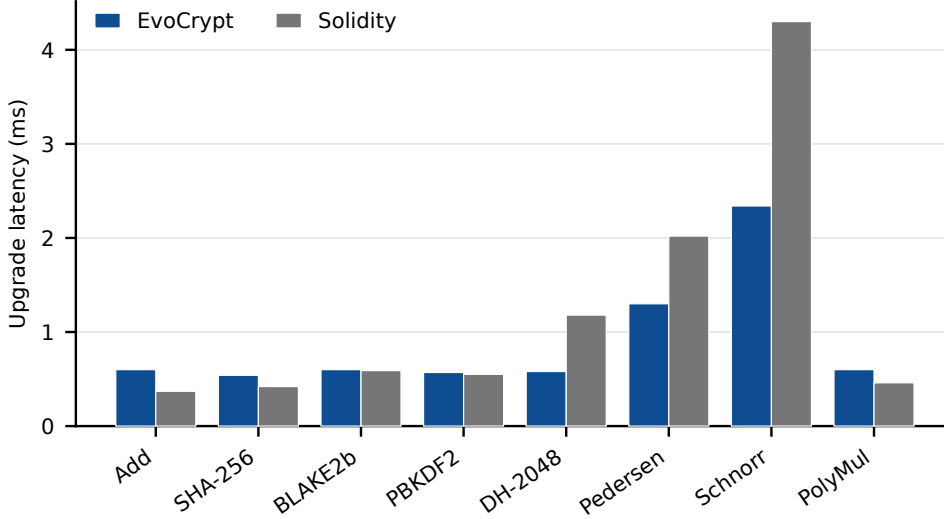


Fig. 3: Setup latency comparison between EvoCrypt upgrade and Solidity contract deployment.

For upgrade efficiency, we compare the coprocessor upgrade upload path with Solidity contract deployment using mean setup latency. For execution efficiency, we compare three paths after setup has completed: WASM-backed Upgrade through CodeStorage (EvoCrypt), Native algorithm, and Solidity Contract. The Native algorithm baseline uses the same Go reference implementations as in Table 2, executed in-process in the evaluation client to measure execution latency outside the EVM rather than through fixed mainnet precompile addresses. Invocation `gasEstimate` is obtained from `eth_estimateGas` on `callFunc` and is not receipt `gasUsed`.

5.2 Upgrade Efficiency

The upgrade-efficiency experiment compared the latency of making a cryptographic algorithm callable through the coprocessor upgrade path against deploying an equivalent Solidity implementation. Figure 3 reports mean setup latency across all eight evaluated algorithms. The Native algorithm baseline is omitted from this comparison because it requires no separate upload or deployment step in the evaluation setup.

On lightweight algorithms, the two setup paths remained close in latency. For Add, SHA-256, BLAKE2b, and PBKDF2, mean upgrade latency stayed below 0.6 ms for EvoCrypt and below 0.6 ms for Solidity, with the largest gap on the non-cryptographic Add baseline (0.60 ms versus 0.37 ms). As algorithm complexity increased, EvoCrypt reduced setup latency relative to contract deployment. For DH-2048, PedersenCommit, and SchnorrVerify, EvoCrypt required 0.58–2.34 ms, whereas Solidity required 1.18–4.30 ms. The largest relative gain appeared on SchnorrVerify, where the upgrade path completed in 2.34 ms compared with 4.30 ms for contract deployment. These latency

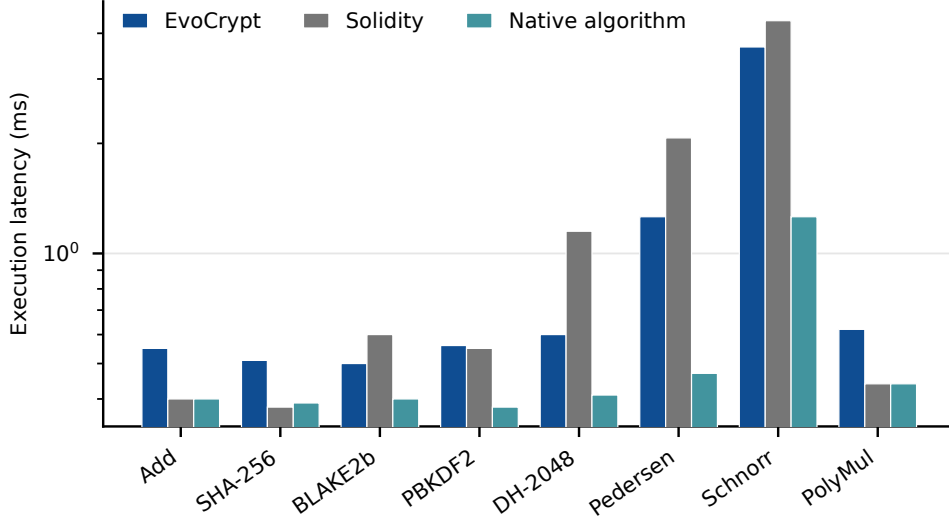


Fig. 4: Execution latency comparison among EvoCrypt, Solidity contract execution, and Native algorithm.

results report network-wide upgrade duration as defined in Section 5.1: Geth-side handling through all evaluated nodes, excluding network propagation and consensus delay. They do not include upgrade-window availability for unrelated EVM traffic.

5.3 Execution Efficiency

The execution-efficiency experiment evaluated steady-state invocation cost after each algorithm had been made callable. Figure 4 compares mean `eth_call` latency across EvoCrypt, Solidity, and Native algorithm paths, and Figure 5 compares `gasEstimate` for EvoCrypt and Solidity. Each path was measured after warm-up calls using repeated `eth_call` samples on the evaluated private-chain prototype.

Native algorithm remained the fastest path on every algorithm, with mean latency between 0.38 ms (PBKDF2) and 1.26 ms (SchnorrVerify). EvoCrypt tracked Native algorithm closely on lightweight workloads: for Add, SHA-256, BLAKE2b, PBKDF2, and PolynomialMul, EvoCrypt latency remained within 0.50–0.62 ms, whereas Solidity stayed within 0.38–0.60 ms on the same set. On heavier cryptographic workloads, EvoCrypt reduced invocation latency relative to Solidity while remaining above Native algorithm. For DH-2048, PedersenCommit, and SchnorrVerify, EvoCrypt required 0.60, 1.26, and 3.67 ms, respectively, compared with 1.15, 2.07, and 4.33 ms through Solidity and 0.41, 0.47, and 1.26 ms through Native algorithm. SchnorrVerify therefore remained the clearest tail case: although EvoCrypt improved on Solidity, it still exceeded Native algorithm by roughly $2.9\times$ on mean `eth_call` latency.

The gas comparison makes the amplification effect directly visible. Solidity executions pay for the expanded EVM instruction sequence, so their estimates grew sharply

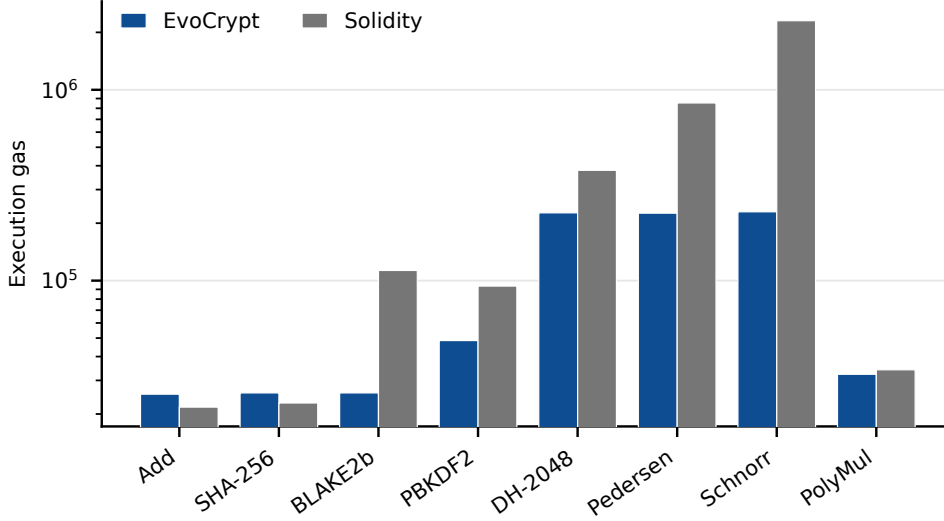


Fig. 5: Invocation `gasEstimate` comparison between EvoCrypt and Solidity contract execution.

with algorithmic complexity: Blake2bSum256, PedersenCommit, and SchnorrVerify required 113,059, 853,145, and 2,301,691 gas through contract execution. EvoCrypt instead charges the registered algorithm-level price, and its estimates for the same workloads were 25,801, 225,695, and 229,437 gas, up to an order of magnitude lower. For Add, SHA-256, and PolynomialMul, whose Solidity expansions remain short, the two contract-facing paths stayed within 13–15% of each other, which is consistent with the amplification account: the separation between the two paths appears exactly where the instruction expansion grows. These numbers describe the current prototype invocation pricing model only; they are not receipt-level `gasUsed` measurements, and the registered prices are experimental parameters rather than a calibrated schedule. The comparison therefore demonstrates the decoupling mechanism and meters the Solidity side, rather than establishing a specific magnitude of gas savings.

6 Related Work

Existing blockchain upgrade schemes primarily replace code at the protocol or client level, where a coordinated version transition preserves a common execution rule across nodes. Cardano’s hard-fork combinator moves the chain between protocol eras at designated upgrade points, while work on updatable blockchains studies synchronization between old and new chains [22–24]. Soft-fork approaches instead place selected policy changes within rules that can be adopted without replacing the entire protocol [25]. Phoenix extends this direction by embedding upgrade code in blockchain transactions and compiling it at runtime. Polkadot stores WASM-based runtime logic on chain and activates governance-approved replacements during block synchronization [10, 26].

These approaches support chain-wide or client-wide evolution, but they require protocol coordination and place the replacement boundary across a substantial portion of system execution.

Contract-level schemes update application logic without replacing the entire blockchain client, using deployable contracts, proxy indirection, or bytecode rewriting. EVMPatch automatically patches deployed Ethereum smart contracts through bytecode rewriting and proxy-based deployment, while Solidity implementations offer application-level flexibility within the EVM [4, 11]. LEAGAN further extends contract-level upgradeability through data separation, incremental hashing, revision control, and status-contract-based version management, enabling contract logic to evolve while preserving persistent state and supporting non-halt deployment [27]. Precompiled Contracts provide a complementary client-native path for selected cryptographic routines through reserved EVM-facing addresses [9, 14, 15]. These mechanisms avoid some forms of client redeployment. Contract-level implementations remain constrained by EVM execution cost and state or interface compatibility, whereas adding or replacing a Precompiled Contract normally requires changes to client and protocol rules.

Dynamic software-update schemes move replacement below the application layer by loading new components into a running host, with emphasis on availability, version compatibility, or isolation. KylinX supports runtime library replacement through dynamic mapping, Mvedsua combines dynamic updates with multi-version execution, PYLIVE changes Python code online, and distributed-system research addresses modular updates and version consistency [19, 20, 28–30]. WASM provides a portable artifact format and explicit host boundary for this style of replacement. In blockchain systems, existing WASM-based upgrades generally target broad runtime logic rather than a narrow EVM-facing cryptographic interface [21, 26]. The Cryptographic Coprocessor addresses this narrower boundary by keeping the EVM-facing call interface stable, preparing replaceable WASM modules asynchronously, and selecting versions through chain-derived metadata and an Activation Block.

7 Conclusion

This paper presented an upgradable Cryptographic Coprocessor for the EVM. The central contribution is to separate a stable contract-facing invocation interface from client-side WASM module implementations, so that cryptographic algorithms can be managed as replaceable modules while ordinary EVM execution remains outside the upgrade mechanism. By routing calls through that interface instead of in-contract Solidity implementations, the design targets both the engineering difficulty and the opcode-level gas amplification that motivate the Solidity baselines evaluated in Section 5, replacing per-instruction metering with the algorithm-level schedule (3). The upgrade protocol combines on-chain metadata (1), off-chain asynchronous WASM preparation, and the height-based selector (2).

The Geth prototype supports the design under the evaluated private-chain scenarios. The upgrade-efficiency experiment showed that publishing algorithms through the coprocessor upload path reduced network-wide upgrade duration—client-side Geth handling only, excluding network propagation and consensus—relative to redeploying

equivalent Solidity implementations on several complex workloads, with SchnorrVerify as the clearest example. The execution-efficiency experiment showed that Native algorithm remained the fastest path, while EvoCrypt stayed within the millisecond range on lightweight algorithms and outperformed Solidity on DH-2048, PedersenCommit, and SchnorrVerify; SchnorrVerify nevertheless remained the clearest tail case relative to Native algorithm. Upgrade consistency remains a protocol property discussed through the Activation Block and forced-upgrade preparation policy rather than a separate experiment.

The current result has clear boundaries. It does not prove zero-downtime service availability, formal WASM sandbox security theorems, behavior when forced preparation cannot complete, state-root equality during upgrades, or consensus safety for arbitrary network sizes. Future work should calibrate the algorithm-level gas price schedule against measured execution cost, harden the upgrade channel against DoS and data-availability saturation attacks, define stricter ABI-version compatibility rules, validate additional cryptographic algorithms and WASM toolchains on further OS and architecture combinations, and design separate consistency and forced-preparation failure evaluations if those properties are later claimed experimentally.

References

- [1] Nakamoto, S.: Bitcoin: A Peer-to-Peer Electronic Cash System. <https://bitcoin.org/bitcoin.pdf>. Accessed 21 August 2026 (2008)
- [2] Zou, W., Lo, D., Kochhar, P.S., Le, X.-B.D., Xia, X., Feng, Y., Chen, Z., Xu, B.: Smart contract development: Challenges and opportunities. *IEEE Transactions on Software Engineering* **47**(10), 2084–2106 (2021) <https://doi.org/10.1109/TSE.2019.2942301>
- [3] Wood, G.: Ethereum: A Secure Decentralised Generalised Transaction Ledger. <https://ethereum.github.io/yellowpaper/paper.pdf>. Ethereum Yellow Paper. Accessed 21 August 2026 (2014)
- [4] Ethereum.org: Ethereum Virtual Machine (EVM). <https://ethereum.org/developers/docs/evm/>. Accessed 21 August 2026
- [5] Shor, P.W.: Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM Review* **41**(2), 303–332 (1999) <https://doi.org/10.1137/S0036144598347011>
- [6] Grover, L.K.: A fast quantum mechanical algorithm for database search. In: *Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing*, pp. 212–219. Association for Computing Machinery, New York, NY, USA (1996). <https://doi.org/10.1145/237814.237866>
- [7] National Institute of Standards and Technology: Post-Quantum Cryptography. <https://csrc.nist.gov/projects/post-quantum-cryptography>. Updated 5 August

2026. Accessed 21 August 2026 (2026)
- [8] Buterin, V.: EIP-150: Gas Cost Changes for IO-Heavy Operations. Ethereum Improvement Proposals, no. 150. <https://eips.ethereum.org/EIPS/eip-150>. Created 24 September 2016. Accessed 21 August 2026 (2016)
 - [9] EVM Codes: Precompiled Contracts. <https://www.evm.codes/precompiled>. Accessed 21 August 2026
 - [10] Wang, C., Li, P., Fan, X., Tang, Z., Zeng, Y., Sakurai, K.: Phoenix: A live upgradable blockchain client. *IEEE Transactions on Sustainable Computing* **8**(4), 703–714 (2023) <https://doi.org/10.1109/TSUSC.2023.3282586>
 - [11] Rodler, M., Li, W., Karame, G.O., Davi, L.: EVMPatch: Timely and automated patching of ethereum smart contracts. In: 30th USENIX Security Symposium (USENIX Security 21), pp. 1289–1306. USENIX Association, Berkeley, CA, USA (2021). <https://www.usenix.org/conference/usenixsecurity21/presentation/rodler>
 - [12] Ethereum.org: Introduction to Smart Contracts. <https://ethereum.org/developers/docs/smart-contracts/>. Updated 25 February 2026. Accessed 21 August 2026 (2026)
 - [13] Ethereum.org: Ethereum Accounts. <https://ethereum.org/developers/docs/accounts/>. Accessed 21 August 2026
 - [14] Reitwiessner, C.: EIP-196: Precompiled Contracts for Addition and Scalar Multiplication on the Elliptic Curve alt_bn128. Ethereum Improvement Proposals, no. 196. <https://eips.ethereum.org/EIPS/eip-196>. Final. Accessed 21 August 2026 (2017)
 - [15] Buterin, V., Reitwiessner, C.: EIP-197: Precompiled Contracts for Optimal Ate Pairing Check on the Elliptic Curve alt_bn128. Ethereum Improvement Proposals, no. 197. <https://eips.ethereum.org/EIPS/eip-197>. Final. Accessed 21 August 2026 (2017)
 - [16] Baylina, J.: EIP-1109: PRECOMPILED CALL Opcode (Remove CALL Costs for Precompiled Contracts). Ethereum Improvement Proposals, no. 1109. <https://eips.ethereum.org/EIPS/eip-1109>. Stagnant. Accessed 21 August 2026 (2018)
 - [17] Ethereum.org: Node Architecture. <https://ethereum.org/developers/docs/nodes-and-clients/node-architecture/>. Updated 25 March 2026. Accessed 21 August 2026 (2026)
 - [18] The go-ethereum Authors: Go Ethereum. <https://geth.ethereum.org/>. Accessed 21 August 2026
 - [19] Ajmani, S., Liskov, B., Shriram, L.: Modular software upgrades for distributed

- systems. In: Thomas, D. (ed.) ECOOP 2006 – Object-Oriented Programming. Lecture Notes in Computer Science, vol. 4067, pp. 452–476. Springer, Berlin, Heidelberg (2006). https://doi.org/10.1007/11785477_26
- [20] Baresi, L., Ghezzi, C., Ma, X., Panzica La Manna, V.: Efficient dynamic updates of distributed components through version consistency. *IEEE Transactions on Software Engineering* **43**(4), 340–358 (2017) <https://doi.org/10.1109/TSE.2016.2592915>
 - [21] World Wide Web Consortium: WebAssembly Core Specification. <https://www.w3.org/TR/wasm-core-2/>. W3C Candidate Recommendation Draft. Accessed 3 September 2026
 - [22] Input Output: Combinator Makes Easy Work of Hard Forks. <https://iohk.io/en/blog/posts/2020/07/23/combinator-makes-easy-work-of-hard-forks/>. Accessed 15 September 2026 (2020)
 - [23] Cardano Foundation: About Hard Forks. <https://docs.cardano.org/about-cardano/evolution/about-hard-forks>. Accessed 15 September 2026
 - [24] Ciampi, M., Karayannidis, N., Kiayias, A., Zindros, D.: Updatable blockchains. In: Chen, L., Li, N., Liang, K., Schneider, S.A. (eds.) *Computer Security – ESORICS 2020*. Lecture Notes in Computer Science, vol. 12309, pp. 590–609. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-59013-0_29
 - [25] Zindros, D.: Soft power: Upgrading chain macroeconomic policy through soft forks. In: Bernhard, M., Bracciali, A., Gudgeon, L., Haines, T., Klages-Mundt, A., Matsuo, S., Perez, D., Sala, M., Werner, S. (eds.) *Financial Cryptography and Data Security. FC 2021 International Workshops*. Lecture Notes in Computer Science, vol. 12676, pp. 467–481. Springer, Berlin, Heidelberg (2021). https://doi.org/10.1007/978-3-662-63958-0_36
 - [26] Polkadot: Runtime Upgrades. <https://docs.polkadot.com/parachains/maintain/runtime-upgrades/>. Accessed 15 September 2026
 - [27] Kumar, G., Saha, R., Conti, M., Buchanan, W.J.: LEAGAN: A decentralized version-control framework for upgradeable smart contracts. *IEEE Transactions on Services Computing* **18**(3), 1529–1542 (2025) <https://doi.org/10.1109/TSC.2025.3562323>
 - [28] Zhang, Y., Crowcroft, J., Li, D., Zhang, C., Li, H., Wang, Y., Yu, K., Xiong, Y., Chen, G.: KylinX: A dynamic library operating system for simplified and efficient cloud virtualization. In: *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pp. 173–186. USENIX Association, Berkeley, CA, USA (2018). <https://www.usenix.org/conference/atc18/presentation/zhang-yiming>

- [29] Pina, L., Andronidis, A., Hicks, M., Cadar, C.: MVEDSUA: Higher availability dynamic software updates via multi-version execution. In: Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems, pp. 573–585. Association for Computing Machinery, New York, NY, USA (2019). <https://doi.org/10.1145/3293882.3330551>
- [30] Huang, H., Xiang, C., Zhong, L., Zhou, Y.: PYLIVE: On-the-fly code change for python-based online services. In: 2021 USENIX Annual Technical Conference (USENIX ATC 21), pp. 349–363. USENIX Association, Berkeley, CA, USA (2021). <https://www.usenix.org/conference/atc21/presentation/huang>